

```
In [233... import numpy as np
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from tensorflow.keras import models, layers
from sklearn.metrics import confusion_matrix, accuracy_score, precision_s
import seaborn as sns
from sklearn.model_selection import train_test_split

import os
```

Question 1

1. Preprocessing

Load the data folder

```
In [234... inside_paths = [os.path.join("fence_renamed/inside", f) for f in os.lis
outside_paths = [os.path.join("fence_renamed/outside", f) for f in os.lis
```

Balance the binary dataset

Done to reduce classification bias

```
In [235... n_samples = min(len(inside_paths), len(outside_paths))
inside_paths = inside_paths[:n_samples]
outside_paths = outside_paths[:n_samples]
print(f"Balanced: {n_samples} per class, {n_samples * 2} total")
```

Balanced: 467 per class, 934 total

Load the actual images

Done using CV2. The images are then converted to grayscale and then thresholded to binary (0 and 1). The image is also resized to reduce the parameters for the model

```
In [236... PREPROCESS_MODE = "binary" # "binary" or "canny"
IMG_SIZE = (64, 64)

original_images = []
images = []
for p in inside_paths + outside_paths:
    original = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    original_images.append(original)

    img = cv2.resize(original, IMG_SIZE)

    if PREPROCESS_MODE == "canny":
        img = cv2.GaussianBlur(img, (5, 5), 0)
        processed = cv2.Canny(img, 100, 200)
    else:
```

```

_, processed = cv2.threshold(img, 180, 255, cv2.THRESH_BINARY_INV

images.append(processed)

images = np.array(images, dtype="float32") / 255.0
images = images[..., np.newaxis] # (N, H, W, 1)

labels = np.array([0] * len(inside_paths) + [1] * len(outside_paths), dtype=

```

Split the data into training and testing sets

```

In [237... train_data, temp_data, train_labels, temp_labels = train_test_split(
    images,
    labels,
    test_size=0.30,
    random_state=42,
    stratify=labels,
)

val_data, test_data, val_labels, test_labels = train_test_split(
    temp_data,
    temp_labels,
    test_size=0.50,
    random_state=42,
    stratify=temp_labels,
)

print(f"Training Data: {len(train_data)}")
print(f"Validation Data: {len(val_data)}")
print(f"Testing Data: {len(test_data)}")

```

Training Data: 653
 Validation Data: 140
 Testing Data: 141

Visualize the Images

```

In [238... fig, axes = plt.subplots(2, 2, figsize=(9, 9))

inside_idx = 0
outside_idx = len(inside_paths)

axes[0, 0].imshow(original_images[inside_idx], cmap="gray")
axes[0, 0].set_title("Inside - Original")
axes[0, 0].axis("off")

axes[0, 1].imshow(images[inside_idx, ..., 0], cmap="gray")
axes[0, 1].set_title(f"Inside - Preprocessed ({IMG_SIZE[0]}x{IMG_SIZE[1]})")
axes[0, 1].axis("off")

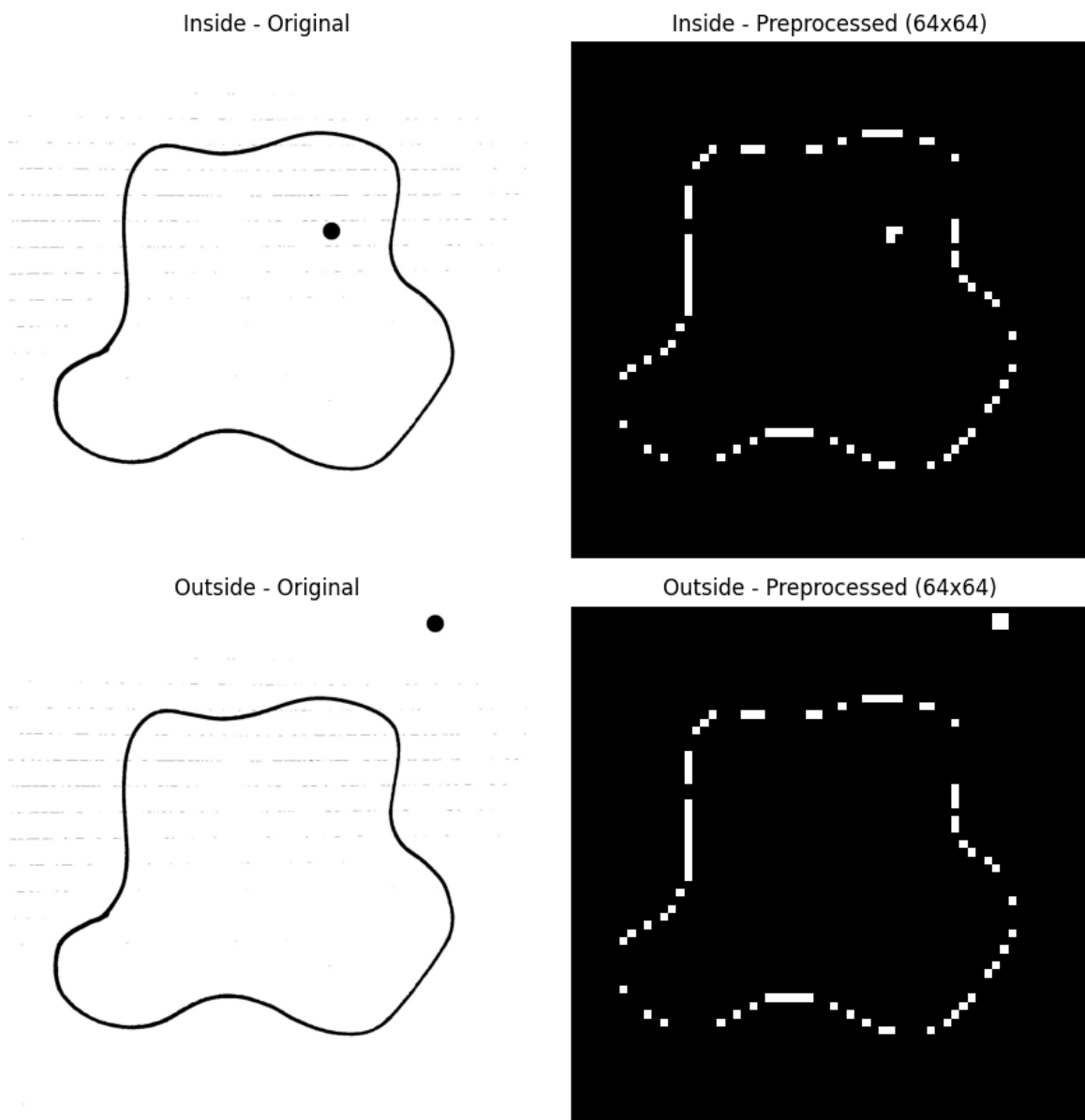
axes[1, 0].imshow(original_images[outside_idx], cmap="gray")
axes[1, 0].set_title("Outside - Original")
axes[1, 0].axis("off")

axes[1, 1].imshow(images[outside_idx, ..., 0], cmap="gray")
axes[1, 1].set_title(f"Outside - Preprocessed ({IMG_SIZE[0]}x{IMG_SIZE[1]})")
axes[1, 1].axis("off")

plt.tight_layout()

```

```
plt.show()
```



2. Constructing the FCN

```
In [239... model = models.Sequential([
    layers.Input(shape=(*IMG_SIZE, 1)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=6,
    min_delta=1e-4,
```

```

        restore_best_weights=True,
    )

    history = model.fit(
        train_data,
        train_labels,
        epochs=80,
        validation_data=(val_data, val_labels),
        callbacks=[early_stop],
        verbose=0,
    )

    test_loss, test_accuracy = model.evaluate(test_data, test_labels, verbose=0)
    print(f'Test accuracy: {test_accuracy}')

    train_loss = history.history['loss']
    train_accuracy = history.history['accuracy']
    val_loss = history.history['val_loss']

```

Test accuracy: 0.9290780425071716

Visualize the Results

```

In [240... fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

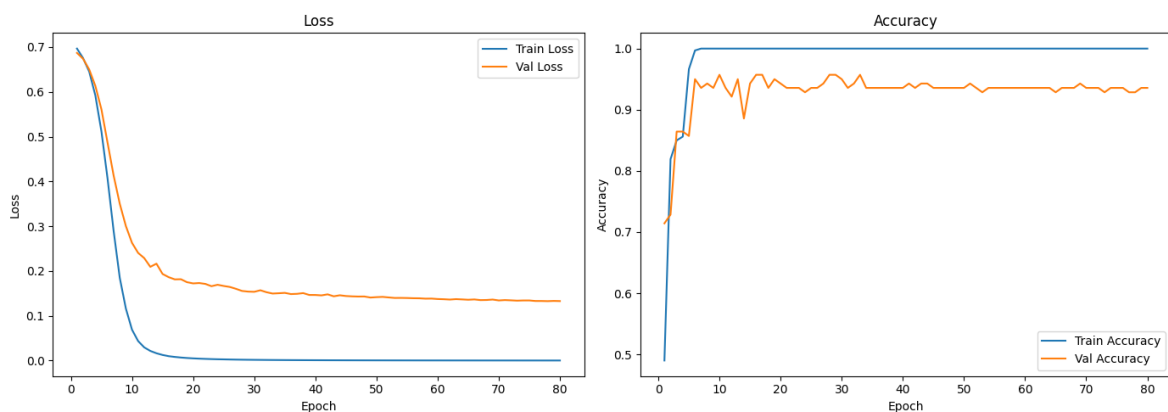
epochs = range(1, len(history.history['loss']) + 1)

ax1.plot(epochs, history.history['loss'], label='Train Loss')
ax1.plot(epochs, history.history['val_loss'], label='Val Loss')
ax1.set_title('Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.legend()

ax2.plot(epochs, history.history['accuracy'], label='Train Accuracy')
ax2.plot(epochs, history.history['val_accuracy'], label='Val Accuracy')
ax2.set_title('Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.legend()

plt.tight_layout()
plt.show()

```



Confusion Matrix

```
In [241... y_pred = model.predict(test_data)
y_pred = (y_pred > 0.5)

cm = confusion_matrix(test_labels, y_pred)
print(cm)

tn, fp, fn, tp = cm.ravel()

labels_cm = [
    [f"TN\n{tn}", f"FP\n{fp}"],
    [f"FN\n{fn}", f"TP\n{tp}"]
]

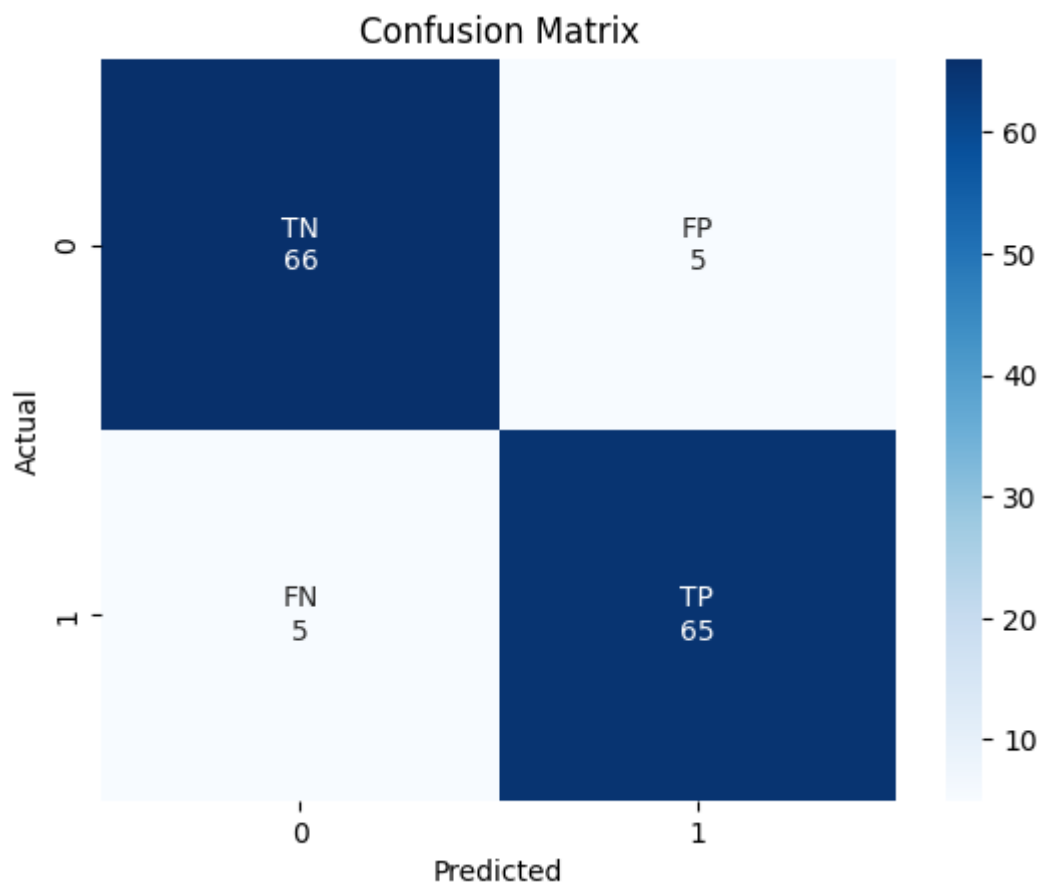
sns.heatmap(cm, annot=labels_cm, fmt='', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

5/5 ————— 0s 35ms/step

```
[[66  5]
 [ 5 65]]
```



Eval Scores

```
In [242... accuracy = accuracy_score(test_labels, y_pred)
precision = precision_score(test_labels, y_pred)
recall = recall_score(test_labels, y_pred)
f1 = f1_score(test_labels, y_pred)

print(f"Accuracy: {accuracy:.4f}")
```

```
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Accuracy: 0.9291
Precision: 0.9286
Recall: 0.9286
F1 Score: 0.9286

Conclusion

The model even though overfitting due to lack of data still learns as we see the training loss and the validation loss decreasing as epochs go on.

The oscillation behaviour in the validation accuracy occurs due to a smaller validation/test dataset as well. Each misclassification changes the accuracy significantly.

In the end we still acquire a good Accuracy given such a simple network and such less dataset

Note: Crucial things in getting high enough accuracy lies in converting the images to binary pixels and balancing out the dataset. plus shuffling given how we are importing the dataset (folder by folder)

Question 2

In [243... `from scipy.linalg import ishermitian`

1. Generating Data

Define validation functions

In [244... `def is_valid(matrix):`

```
    eigenvalues = np.linalg.eigvals(matrix)
    trace = np.trace(matrix)

    cond1 = ishermitian(matrix)
    cond2 = np.all(eigenvalues >= 0)
    cond3 = np.isclose(trace, 1)

    return cond1 and cond2 and cond3

def make_valid_matrix():
    A = np.random.randn(2,2)
    rho = A @ A.conj().T
    rho = rho / np.trace(rho)

    return rho
```

Generate Data

```
In [245... valid_matrices = []
invalid_matrices = []

for _ in range(5000):
    matrix = make_valid_matrix()
    valid_matrices.append(matrix)

done = False
while not done:
    matrix = np.random.randn(2,2)

    if not is_valid(matrix):
        invalid_matrices.append(matrix)

    if len(invalid_matrices) >= 5000:
        done = True
```

Split the data for training and testing

```
In [246... matrix_data = np.concatenate([valid_matrices, invalid_matrices])
matrix_labels = np.concatenate([np.ones(len(valid_matrices)), np.zeros(len(
train_data_matrix, test_data_matrix, train_labels_matrix, test_labels_mat

print(f"Training Data: {len(train_data)}")
print(f"Testing Data: {len(test_data)}")
```

Training Data: 653

Testing Data: 141

2. Training the Neural Net

```
In [247... model_matrix = models.Sequential([
    layers.Input(shape=(2,2)),
    layers.Flatten(),
    layers.Dense(8, activation='relu'),
    layers.Dense(4, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model_matrix.compile(optimizer='adam',
                    loss='binary_crossentropy',
                    metrics=['accuracy'])

history_matrix = model_matrix.fit(
    train_data_matrix,
    train_labels_matrix,
    epochs=50,
    validation_data=(test_data_matrix, test_labels_matrix),
    verbose=0
)

test_loss, test_acc = model_matrix.evaluate(test_data_matrix, test_labels)
print(f'Test accuracy: {test_acc}')
```

```

train_loss = history.history['loss']
train_accuracy = history.history['accuracy']
val_loss = history.history['val_loss']

```

63/63 ————— 0s 2ms/step - accuracy: 0.9990 - loss: 0.0038
 Test accuracy: 0.9990000128746033

```

In [248... fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

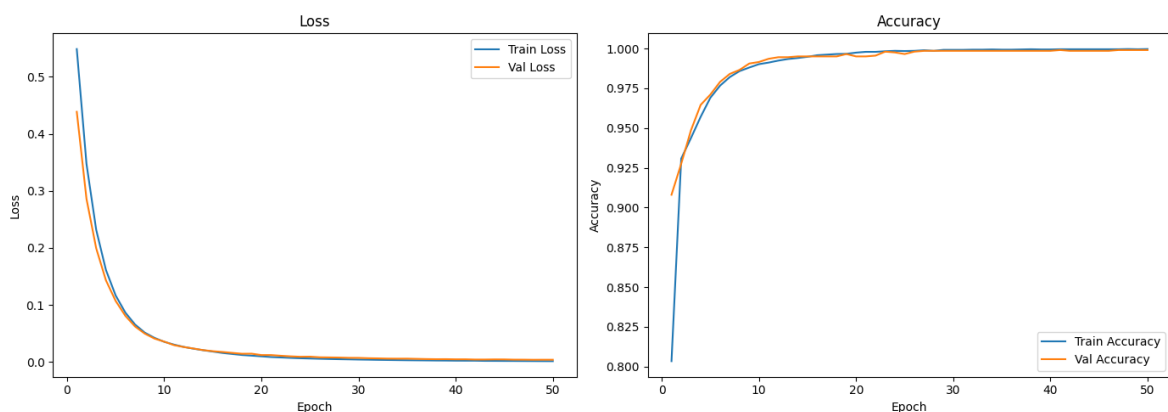
epochs = range(1, len(history_matrix.history['loss']) + 1)

ax1.plot(epochs, history_matrix.history['loss'], label='Train Loss')
ax1.plot(epochs, history_matrix.history['val_loss'], label='Val Loss')
ax1.set_title('Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.legend()

ax2.plot(epochs, history_matrix.history['accuracy'], label='Train Accuracy')
ax2.plot(epochs, history_matrix.history['val_accuracy'], label='Val Accuracy')
ax2.set_title('Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.legend()

plt.tight_layout()
plt.show()

```



```

In [249... y_pred_matrix = model_matrix.predict(test_data_matrix)
y_pred_matrix = (y_pred_matrix > 0.5)

cm = confusion_matrix(test_labels_matrix, y_pred_matrix)
print(cm)

tn, fp, fn, tp = cm.ravel()

labels_matrix = [
    [f"TN\n{tn}", f"FP\n{fp}"],
    [f"FN\n{fn}", f"TP\n{tp}"]
]

sns.heatmap(cm, annot=labels_matrix, fmt='', cmap='Blues')

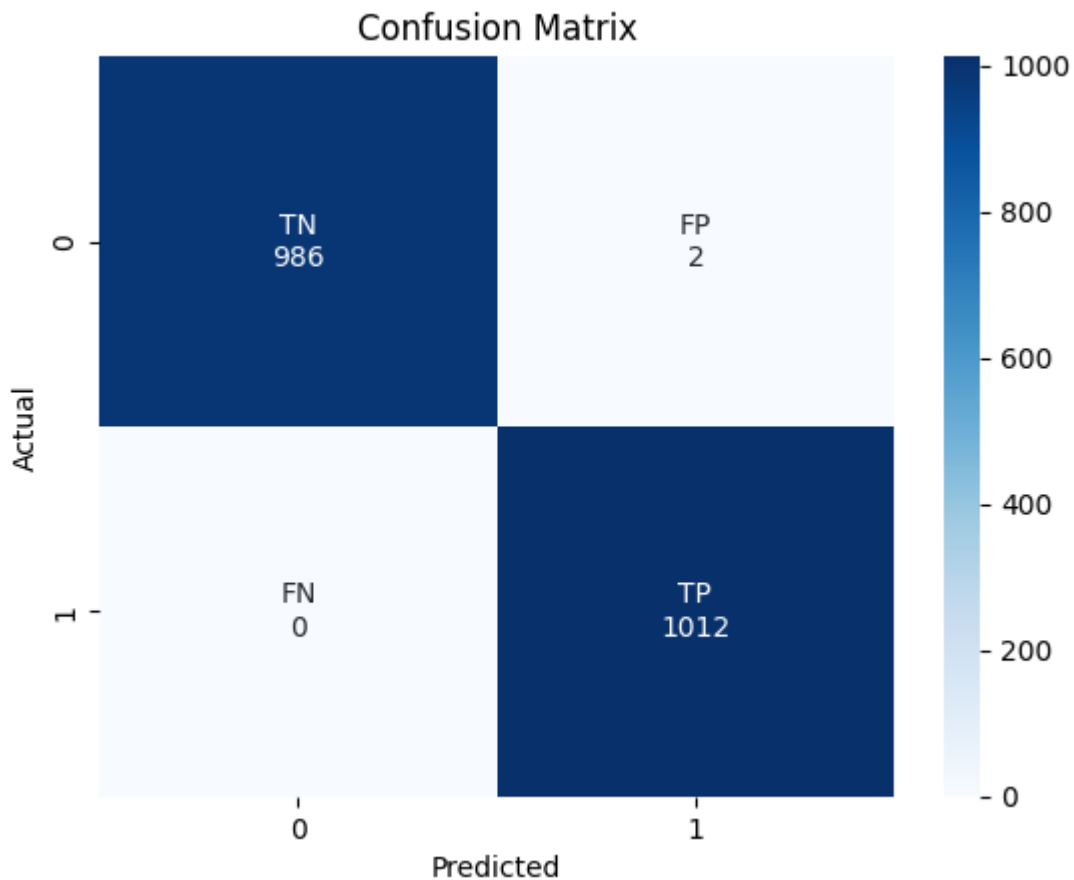
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

```

```
plt.show()
```

63/63 ————— 0s 3ms/step

```
[[ 986    2]
 [   0 1012]]
```



```
In [250... accuracy = accuracy_score(test_labels_matrix, y_pred_matrix)
precision = precision_score(test_labels_matrix, y_pred_matrix)
recall = recall_score(test_labels_matrix, y_pred_matrix)
f1 = f1_score(test_labels_matrix, y_pred_matrix)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Accuracy: 0.9990
Precision: 0.9980
Recall: 1.0000
F1 Score: 0.9990

Even though the training is overfitting we still get 99% accuracy and good training and validation loss landscapes. This just shows how easy is the problem for FCN to solve

3. Model Evaluation

```
In [251... eval_data = []
eval_labels = []
```

```
for _ in range(500):
    matrix = make_valid_matrix()
    eval_data.append(np.array(matrix))
    eval_labels.append(1)

done = False
while not done:
    matrix = np.random.randn(2,2)
    eval_data.append(np.array(matrix))
    eval_labels.append(0)

    if len(eval_data) >= 1000:
        done = True

eval_data = np.array(eval_data)
eval_labels = np.array(eval_labels)

y_pred_matrix = model_matrix.predict(eval_data)
y_pred_matrix = (y_pred_matrix > 0.5)

cm = confusion_matrix(eval_labels, y_pred_matrix)
print(cm)
```

32/32 ————— 0s 5ms/step

```
[[497   3]
 [   0 500]]
```

Thus for the generated data where 500 valid and 500 invalid matrixes exist, we get the above confusion matrix which shows our model is highly accurate